PART A

a) Teams dynamically selects the best mode based on real-time network conditions, in order to balance performance and provide the best possible user experience.

Mode	Transport Layer	Session Layer	Presentation Layer	Application Layer
1-UDP	UDP-connectionless is used to provide low latency and fast delivery.	• Sender sends packets to receiver without formally establishing a connection. • Does not maintain connection state making it ideal for video/audio	• Translation of data into understandable format for transmission. • Handles encoding and compression of audio data.	Teams sends/receives audio packets directly.
2-UDP via Supernode	UDP-connectionless but via a relay (Supernode).	• Teams establishes-manages session through a relayed channel. • Teams tracks packet delivery via Acknowledgements to handle packet loss.	• Translation of data into understandable format for transmission. • Handles encoding and compression of audio data.	Teams behaves similarly to Mode 1, but sends UDP packets to a Supernode. Supernode forwards these to the destination. Selective acknowledgements are relayed back via the Supernode.
3-TCP then UDP via Supernode	TCP-connection oriented is used from sender to	Teams establishes-manages session, uses TCP to ensure reliable	Handles encoding and compression of audio	Teams connects to a Supernode over TCP.

	Supernode, then UDP-connectionless from Supernode to receiver.	delivery to Supernode, then uses UDP from Supernode to receiver with selective acknowledgements	for both TCP and UDP protocols.	Audio packets are tunnelled via TCP to the Supernode. Supernode forwards them over UDP to the receiver. Selective acknowledgements are used to optimize delivery.

b)

Mode	**Audio/video problems**
1-UDP	• Audio dropouts, video freezes due to unreliable delivery (UDP-connectionless).
3-TCP then UDP via Supermode	• Long freezes and delayed video output (video-audio mismatch) due to TCP nature. • Robotic voice due to high latency.

c) As we learned in class UDP-connectionless is low latency and faster because it doesn't wait for acknowledgments. I guess the correct answer is based on the scenario(network conditions, working environment). For me **mode 1-Direct connection using UDP** seems like the best mode because it offers faster data transmission for time-sensitive use cases such as audio and video.

d) ATM is a high-speed multiplexing and switching method utilizing fixed-length cells of 53 octets to support multiple types of traffic. ATM is fundamentally a connection-oriented (PVC, SVC). While ATM does not provide built-in error control or acknowledgements, selective acknowledgments would improve overall user experience and more specifically:

1.Reliability

- Selective acknowledgements ensure consistent audio/video quality.

2. Delay Variations

- ATM is designed for low-latency and low-jitter transmission, especially using CBR (Constant Bit Rate) or RT-VBR (Real-Time Variable Bit Rate) service categories. These are ideal for real-time applications like Teams.
- ATM's fixed cell size (53 bytes) helps reduce queuing and switching delays compared to variable-length IP packets.

Thus, Teams would likely experience **less jitter and more consistent interarrival times** using an ATM backbone, particularly if RT-VBR is used.

3. Interarrival Delays

- Because ATM transmits data in small, fixed-size cells, the interarrival time between cells is more predictable.

In theory in an ATM network with selective acknowledgments, Teams would likely experience less jitter and more consistent interarrival times using an ATM backbone, particularly if RT-VBR is used. Teams would benefit from reduced delay variation, better packet interarrival regularity, and predictable performance, especially for audio and video streams.

PART B

1.

	Performance	Reliability
Device A	• Tunnelling means packets take a less efficient path to their destination. • Need to add VPN headers to packets, increasing the total packet size.	• VPN connections can drop unexpectedly due to mobile network handoffs. • If the VPN server is under high load it may cause packet loss or disconnections.
Device B	No header encapsulation or Tunnelling which means: • Smaller-faster packets. • Shorter path to server	

Nowadays the biggest Apps in the market use cloud servers rather than a true P2P, which helps mitigate any **combability** issues . Also, there is consent from the owner . This leads me to believe there will be no **combability** issues.

2. In terms of security ,these Apps use End-to-End Encryption, which protects the contents of the messages. VPN or no VPN the data remains protected.

3. End-to-End encryption provides confidentiality and message integrity, making sure no attacker or the Application owner himself can read the contents of a message, whether its from Device A or Device B.

4. The App might suspect unauthorized access to an account. It will notify the main device if he authorises this login attempt and the main device will need to give access.

5. Steps developers can take to ensure robust reliability and security when designing multi-device synchronization features:

1. Use End-to-End Encryption (E2EE)

- Encrypt all data per device, so even if synced through the cloud or VPN, content remains secure.

2. Device Identity Management

- Require explicit user approval for adding new devices (e.g., QR code scan)
- Allow users to view and delete connected devices at any time.

3. Use a cloud-mediated architecture rather than P2P to handle:

- NAT traversal
- VPN-induced IP changes

4. Good Practises

- Avoid sending unnecessary large files or metadata repeatedly.

6. Encapsulation adds extra headers, this increases each packet's total size. Because the data is tunnelled the traffic goes through the VPN server first, not directly to the app's servers. This increases latency and creates an inefficient routing path. Tunnelling also creates a single point of failure- the VPN session. If the tunnel drops, The device's internet connection is dropped. Since the IP is masked ,It may cause security alerts or login denials.